The Chinese University of Hong Kong, Shenzhen

CSC 3170

Database System

Report for Project

Author	Student ID
韩炅皓 HAN-Jionghao	119010087
胡海川 HU-Haichuan	119010103
李韵文 LI-Yunwen	119010163
邵晓雯 SHAO-Xiaowen	119010258
赵松霖 ZHAO-Songlin	120090346

May 14, 2022

Contents

1 Introduction and motivation **2**

2 Design and implementation **2**

2.1 Design . . . 2

2.1.1 Business Requirements . . . 2

2.1.2 ER Diagram . . . 2

2.1.3 Relation Schema . . . 4

2.2 Implementation . . . 6

2.2.1 Realistic Data . . . 6

2.2.2 Maintenance of database . . . 6

2.2.3 Sample CRUD queries of daily operations . . . 6

2.2.4 Index . . . 9

3 Data analysis and data mining **9**

3.1 Data Analysis . . . 9

3.2 Data Mining . . . 10

4 Conclusion **12**

5 Self-evaluation **12**

6 Appendices **13**

6.1 Toolchain . . . 13

6.2 GitHub Repo . . . 13

6.3 Build and run . . . 13

6.4 Demo . . . 13

6.5 Contribution . . . 13

References **14**

1 Introduction and motivation

Vocabulary plays a fundamental role in preparing for further education and standardized English tests (Kinkade, 2019; Al-Jarf, 2022). Users could use the vocabulary builders to enrich their vocabulary repertoire, learn faster, and memorize words better (Kinkade, 2019; Al-Jarf, 2022). Users also get more enthusiastic when learning vocabulary using digital flashcards (Setiawan & Wiedarti, 2020). Apps in the market, including GRE3000 Words and BaiCiZhan, are usually free, handy, and easy to download. These apps help students prepare for the IELTS, TOEFL, and GRE tests. Students can use these apps anywhere and anytime they want, without restricting themselves with some physical copies of word books and dictionaries. Students could utilize their brief scraps of spare time to recite the words on an app without finding a complete chunk of time for memorizing.

This inspired us to implement this project: Design a database for a word-memorizing web app: Vocabulary3170. We also created a web app to access and manage our database.

2 Design and implementation

2.1 Design

2.1.1 Business Requirements

This project identifies two account types: Standard users and Administrator accounts.

All users should be able to log in to their accounts and memorize the words with our web app. Each user account has information, including a username, password, email, and phone number. They could then browse the word books and choose a word book to recite. After choosing a word book, users can browse vocabulary items alphabetically or sequentially. Users study the vocabularies in a word book by swiping the word cards and viewing their spelling and meanings. The presses on the "known" (i.e., a user knows this word) and the "unknown" (i.e., a user does not know this word) buttons are tracked. Also, users could set their daily goals on how many words are about to memorize in a day. The progress on the goals is tracked.

Administrators should be able to create, delete, and update word books. When a new word is inserted, they could also update vocabularies in a word book. They have full control and complete access to our database, which has higher privilege compared with standard users who have only limited access to our database.

2.1.2 ER Diagram

We need to determine the E-R diagram and schema to design the database. To construct the database better, we do the normalization to make it satisfy the requirements for the 4NF.

This is the E-R diagram we designed based on the app‘s requirements and the analysis of the relational schema. We identify five entities: users, word books, vocabularies in a book, evaluation reports of users‘ progress, and evaluation reports of how well a user memorizes a word, which is shown in 1.

- **Entity Set**

User: Store information of our web-app users. Each user has a unique user id, a username, password, a boolean variable indicating whether the user is an administrator, and other personal information.

Book: Store the vocabulary books in our web app. Each book possesses a unique book id, a book name, and its word count.

Word: A weak entity of "book" composed of word id, spelling, and translation.

Plan_evaluation: Keep track of users' goals and progress on a book in daily records. Its attributes include "daily_goal", "daily_progress", and "date".

Recite_evaluation: Keep track of users' grasp of words daily. The fields "correct" and "error" record the users' number of presses on the "known" and "unknown" buttons below a word.

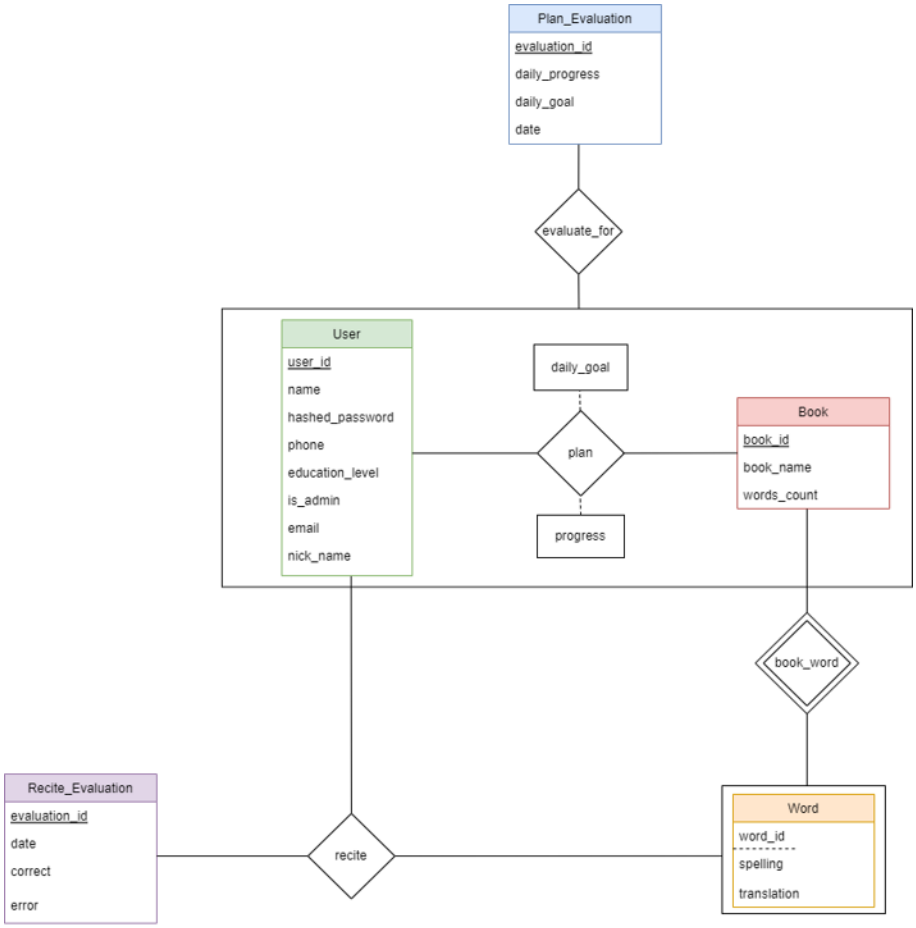

Figure 1: ER Diagram of our database system

- **Relationship Set**

Plan: This is a binary relation between the "user" and "book", named "plan". We have the descriptive attributes 'daily_goal' and 'progress' attached to the relationship set 'plan.'

Daily_goal: This relation describes users' current daily goals.

Progress: This relation describes the number of vocabularies a user has recited in a day.

Book_word: This relation represents the association between a word book and its vocabulary.

Evaluate_for: This relation describes evaluating users‘ daily progress on a word book, which is an aggregated relationship between an evaluation report ‘plan_evaluation‘ and ‘plan.‘

Recite: This relation is the association between a user, a vocabulary, and an evaluation report.

2.1.3 Relation Schema

We need to reduce the ER diagram into the relational schema to help us develop the tables and check the dependencies.

- For example, the entity set ‘word,‘ which is a weak entity. So we can ignore the ‘book_word‘ relationship set while adding the primary key of the book entity set.
- For another example, since the relation between the entity sets ‘user‘ and ‘book‘ is the many-to-many relationship, the primary key for the ‘plan‘ relationship set consists of the primary key of the entity set ‘book‘ and entity set ‘word.‘

After checking the dependencies between attributes one by one, we can get the following relational schema:

User (user_id, name, hashed_password, email, phone, education, is_admin)

Book (book_id, name, words_count)

Word (book_id, word_id, spelling, translation)

Plan (user_id, book_id, progress, daily_goal)

Plan_evaluation (user_id, book_id, evaluation_id)

Plan_evaluation_detail (evaluation_id, date, daily_progress, daily_goal)

Recite_evaluation (user_id, book_id, word_id, evaluation_word_id)

Recite_evaluation_detail (evaluation_word_id, date, error, correct)

And we can also obtain the following figure.

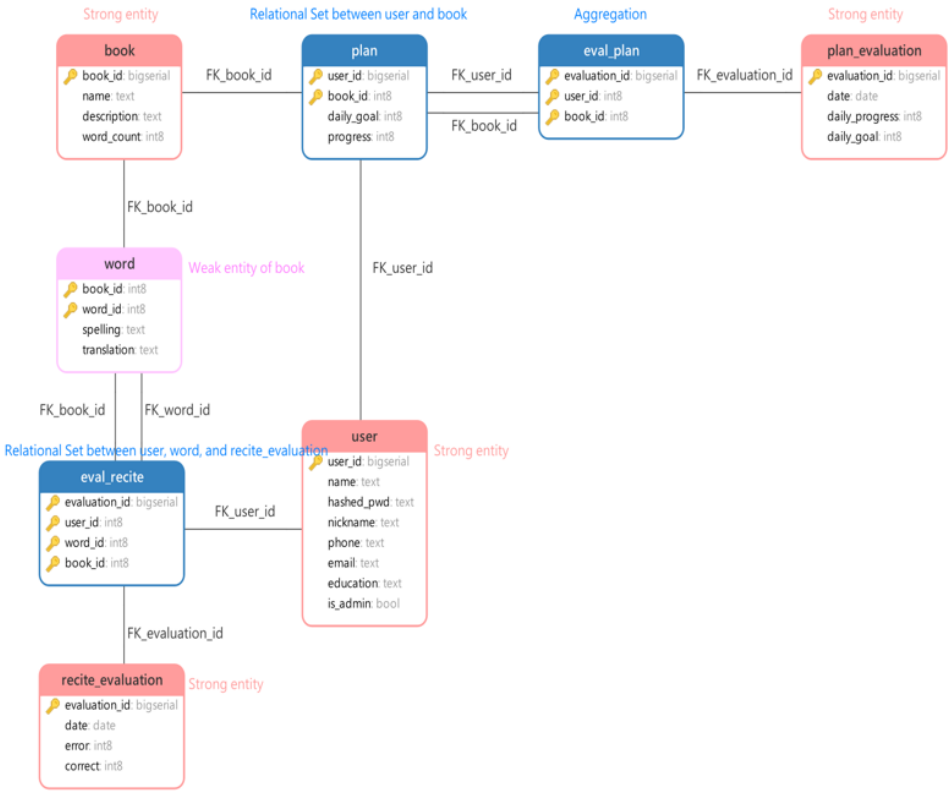

Figure 2: The relational schema of our database system

- **Primary Keys and Foreign Keys** For each entity, the primary keys are indicated with key icons. For example, in the entity 'book', we have the 'book_id' as the primary key. Foreign keys are indicated with lines annotated with the name of the foreign key with the prefix "FK_". For example, 'book_id' is a foreign key from 'word' referencing 'book,' shown in the Figure2.

- **Non-trivial Functional Dependencies**

  user_id → name, hashed_password, email, phone, education, is_admin

  book_id → name, words_count

  book_id , word_id → spelling, translation

  user_id, book_id → progress, daily_goal

  evaluation_id → date, daily_progress, daily_goal

  evaluation_word_id → date, error, correct

- **Normalization** Our relational schema is in 4 NF naturally. Firstly, it is in BCNF where every determinant is a superkey. Moreover, we do not have any non-trivial MVDs, even in the plan_evaluation and recite_evaluation schema.

2.2 Implementation

2.2.1 Realistic Data

We find some vocabulary books online and populate them into our database.

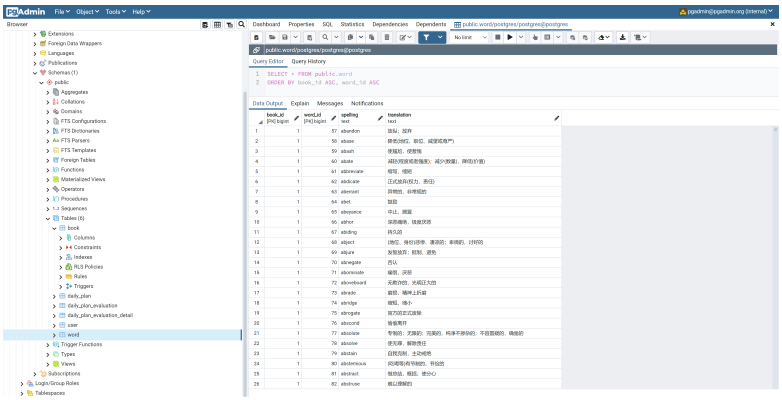

Figure 3: Realistic data we import

2.2.2 Maintenance of database

We used some methods to maintain the quality of our data:

- We achieved atomic update. We use the *with* statement in Python as follows:

```
async def create(self):
    async with self._connection_generator() as conn:
        async with conn.cursor() as cur:
            await cur.execute(
                '''SQL_STATEMENT'''
            )
```

Program 1: Atomic update

  This guarantees that each update is atomic even if multiple SDL queries are in one update.

- All foreign keys are constrained as CASCADE ON DELETE. Therefore, if we delete a referenced entity, all the referencing entities will also be deleted.
- TRIGGER is used to guarantee the integrity of the data constraints and business rules. For example, on insertion of a word, its word_id should increase automatically, and the corresponding book.word_count should also be updated.
- To manage users' authority on the data, we used the field user.is_admin. The default value is FALSE.

2.2.3 Sample CRUD queries of daily operations

We firstly create the tables. The query for creating "word" is shown below:

```sql
CREATE TABLE IF NOT EXISTS "word"(
    "book_id" BIGINT NOT NULL REFERENCES "book"("book_id"),
    "word_id" BIGINT NOT NULL,
    "spelling" TEXT NOT NULL,
    "translation" TEXT,
    PRIMARY KEY ("book_id", "word_id")
);
CREATE OR REPLACE FUNCTION "fill_in_word_seq"()
RETURNS TRIGGER
LANGUAGE PLPGSQL
AS $$
BEGIN
    NEW."word_id" := nextval('"book_seq_' || NEW.book_id || '"');
    RETURN NEW;
END
$$;
CREATE OR REPLACE TRIGGER "fill_in_word_seq"
BEFORE INSERT ON "word"
FOR EACH ROW EXECUTE PROCEDURE "fill_in_word_seq"();
CREATE OR REPLACE FUNCTION "update_words_count_after_insert"()
RETURNS TRIGGER
LANGUAGE PLPGSQL
AS $$
BEGIN
    UPDATE "book"
    SET "words_count" = "words_count" + 1
    WHERE "book_id" = NEW."book_id";
    RETURN NEW;
END
$$;
CREATE OR REPLACE TRIGGER "update_words_count_after_insert"
AFTER INSERT ON "word"
FOR EACH ROW EXECUTE procedure "update_words_count_after_insert"();
CREATE OR REPLACE FUNCTION "update_words_count_after_delete"()
RETURNS TRIGGER
LANGUAGE PLPGSQL
AS $$
BEGIN
    UPDATE "book"
    SET "words_count" = "words_count" - 1
    WHERE "book_id" = OLD."book_id";
    RETURN OLD;
END
$$;
CREATE OR REPLACE TRIGGER "update_words_count_after_delete"
AFTER DELETE ON "word"
FOR EACH ROW EXECUTE procedure "update_words_count_after_delete"();
```

Program 2: Query for creating table “word“

We also write a bunch of CRUD queries that our app needs for daily operations. Some of them are listed below:

- C: Insert a reciting plan based on the book_name and user_id

```python
await cur.execute(
    f'''
        INSERT INTO "daily_plan"(
            "user_id",
            "book_id",
            {', '.join([f'"{key}"' for key in adding_daily_plan_dict.keys()])}
        )
        VALUES(
            %s,
            %s,
            {', '.join(['%s'] * len(adding_daily_plan_dict))}
        )
        RETURNING *;
    ''',
    [
        user_id,
        book.book_id,
        *adding_daily_plan_dict.values(),
    ],
)
```

Program 3: Query for inserting a plan

- R: retrieve a word by word_id and book_id

```python
await cur.execute(
    '''
        SELECT * FROM "word"
        WHERE "book_id" = %s
        AND "word_id" = %s;
    ''',
    [
        book_id,
        word_id,
    ],
)
```

Program 4: Query for word retrieval

- U: Update a user's hashed password

```python
if password_hasher.check_needs_rehash(user.hashed_password):
    user.hashed_password = password_hasher.hash(password)
    await cur.execute(
        '''
            UPDATE "user"
            SET "hashed_password" = %s
            WHERE "user_id" = %s
            RETURNING *;
        ''',
        [
            user.hashed_password,
            user.user_id,
        ],
    )
```

Program 5: Query for updating password hash

- D: delete a book

  All of its words will also be deleted due to the CASCADE ON DELETE constraint.

```
await cur.execute(
    '''
        DELETE FROM "book"
        WHERE "name" = %s
        RETURNING *;
    ''',
    [
        name,
    ],
)
```

Program 6: Query for deleting a book

2.2.4 Index

To increase performance, we used the B-tree index. In Postgresql, all UNIQUE or PRIMARY KEY constrained columns are automatically indexed in B-tree. Besides primary keys and unique values like the user's phone, we mainly index the date fields because we frequently search through this field during data analysis and word reciting. B-tree is preferred to hash because we may need to search in range, group, and order.

```
CREATE INDEX IF NOT EXISTS "plan_date_index"
ON "daily_plan_evaluation_detail"
USING btree (
    "date"
);
```

Program 7: Query for creating index on daily_plan_evaluation_detail.word

3 Data analysis and data mining

3.1 Data Analysis

A vocabulary-building app is essential to provide visualizations of reciting statistics. As a result, we provide visualizations of users' learning outcomes to be aware of how they perform in a particular period. Users can see the number of recognized and forgotten words each day in a bar chart. Here we provide sample queries for retrieving the number of recognized words and forgotten words of a user in a day.

```
SELECT COUNT(*) FROM
↪  "recite_evaluation", "eval_recite"
WHERE
↪  "recite_evaluation"."evaluation_id"
    = "eval_recite"."evaluation_id"
AND "date" = date
AND "user_id" = user_id
AND "correct" = 1
AND "error" = 0;
```

```
SELECT COUNT(*) FROM
↪  "recite_evaluation", "eval_recite"
WHERE
↪  "recite_evaluation"."evaluation_id"
    = "eval_recite"."evaluation_id"
AND "date" = date
AND "user_id" = user_id;
```

Figure 4: SQL queries for getting the number of recognized and forgotten words

By retrieving a user's data from May 1st to May 7th, we can plot a bar chart of the number of recognized and forgotten words in this period. In addition, the proportion of the recognized

words among all the recited words each day can also be read from the diagram.

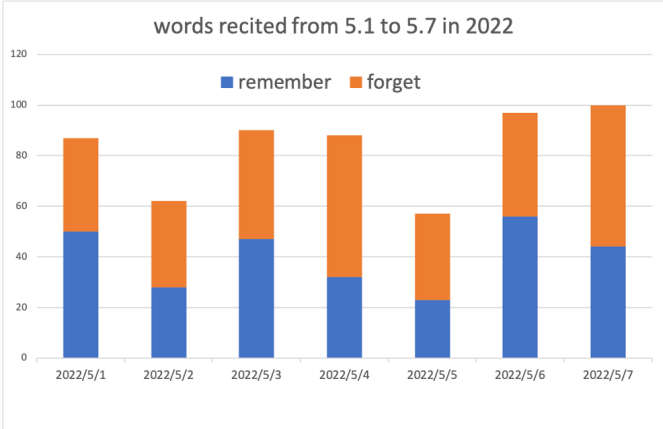

Figure 5: user's performance during 5.1 to 5.7 in 2022

3.2 Data Mining

Furthermore, we can extract some useful information from users' data in the database. A common sense is that the learning ability is different for users from diverse educational backgrounds. In our project, we randomly generated user's data by $\mathbb{P}(X = j) = \alpha p_j^{(1)} + (1 - \alpha)p_j^{(2)}, j \leq 0$, where $0 < \alpha < 1$. $p_j^{(1)}$is the pmf of a multivariate normal distribution $N_2(u, \Sigma)$ and $p_j^{(2)}$ is the pmf of a multivariate uniform distribution $U\left((a, b)^n\right)$.

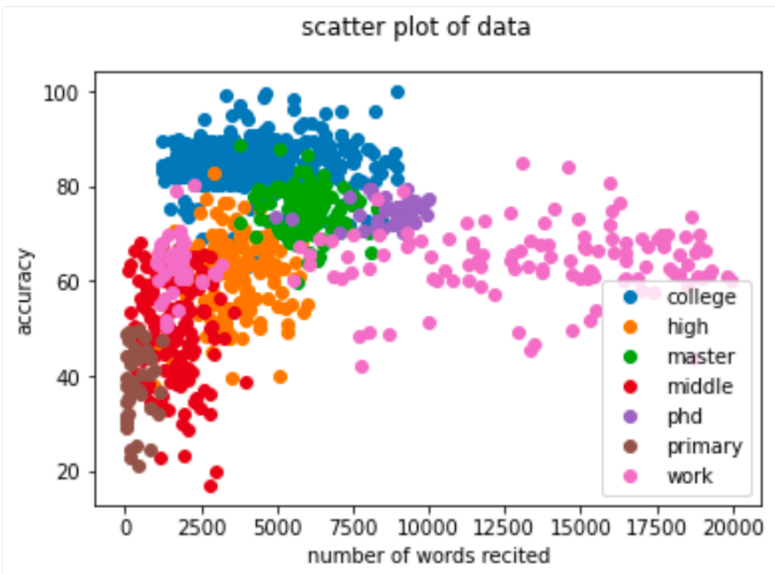

Figure 6: scatter plot of the number of words recited and reciting accuracy of all users

As shown in the scatter plot, the data contains randomness to simulate the actual life data better and is still separable. In this case, we can predict a user's education level by his/her learning ability which is measured by the total number of words recited in the software and reciting accuracy. The total number of words recited is defined as the number of words a user encounters. Recording accuracy is the proportion of words that a user recognizes when a user

meets the words for the first time. We use the decision tree model to make the prediction. In this task, we use the Gini index to determine impurity, and other parameters of the decision tree are determined by cross-validation. The resulting decision tree is shown below.

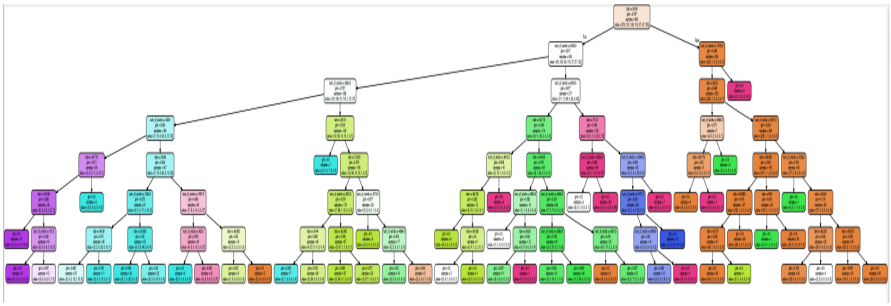

Figure 7: The decision tree of predicting a user's education level

The SQL queries to get a user's total number of words recited and reciting accuracy are as follows. The first query gets the total number of words recited by a user, and the second query retrieves the number of recognized words of a user. The reciting accuracy then is $\alpha = \frac{\text{result of } q_2}{\text{result of } q_1}$.

```
SELECT "user_id", COUNT(DISTINCT
↪  "book_id", "word_id") FROM
↪  "eval_recite"
GROUP BY "user_id"
ORDER BY "user_id" ASC;
```

```
 SELECT "user_id", SUM(CASE WHEN "error" = 0 THEN 1 ELSE 0 END) FROM
(
SELECT "user_id", MIN("date"), "book_id", "word_id", "error" FROM
↪  "recite_evaluation", "eval_recite"
WHERE "recite_evaluation"."evaluation_id" =
↪  "eval_recite"."evaluation_id"
GROUP BY "user_id", "book_id", "word_id"
) AS "user_first_see_word"
GROUP BY "user_id"
ORDER BY "user_id" ASC;
```

Figure 8: Relevant SQL queries

Our decision tree classifier will output the predicted education level by inputting data into the decision tree. The decision boundary is shown below to visualize better how the decision tree classifies data.

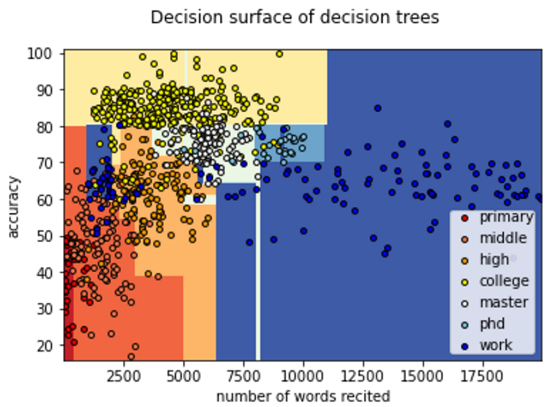

Figure 9: the decision boundary of our model

The data mining function of our app can also provide further recommendations to users. Suppose a user recites data that falls in the range of yellow, then we cal predict that he/she is a university

student. Then we can further recommend some books that may be useful for university students like CET-4 or CET-6.

4 Conclusion

In conclusion, we designed a database for a vocabulary-building app. We designed the ER diagram and reduced it to normalized relational schema. We populate our schema with some realistic data from the word books. Using the *with* statement of python, atomic updates are realized. The integrity of data is also maintained via various methods. We also wrote several SQL queries to facilitate our app's daily operations and activities, visualize users' learning outcomes, and predict their education levels. We indexed unique data fields or served as primary keys using B-tree to increase performance. All the operations could be shown via our web app demo (See demo video in Section 6.4 and codes in Section 6.2).

5 Self-evaluation

In this project, we learned to think from both the customers' view and a database engineer's view. It helps us better understand the actual implementation of a database and the absolute need for a database. We carefully designed the E-R diagram to fit the requirements. The structure is both transparent and efficient. We maintained 4NF with little harmful redundancy. Besides, we set the foreign keys and relationships based on dependency. Indexing was also applied to improve data access performance. The database was populated with realistic data crawled from different vocabulary books. The GUI of the system is simple, pretty, and user-friendly. We clearly illustrated operational and analytical SQL query functions, including objectives, SQL sentences, and SQL results.

Last but not least, we performed data mining to explore the relationship between education level users' reciting performance using the decision tree model. This part is a highlight for the bonus. We may add more data and attributes to expand and extend our database for further improvement. More functions such as cross-module query can be implemented. The GUI can become fancier.

6 Appendices

6.1 Toolchain

Our toolchain is shown in Figure 10. Our backend server is based on Python, which reads and executes backend codes to send queries to the PostgreSQL database. Our frontend program packs static files from frontend codes. And we use the browser to send URL to the server.

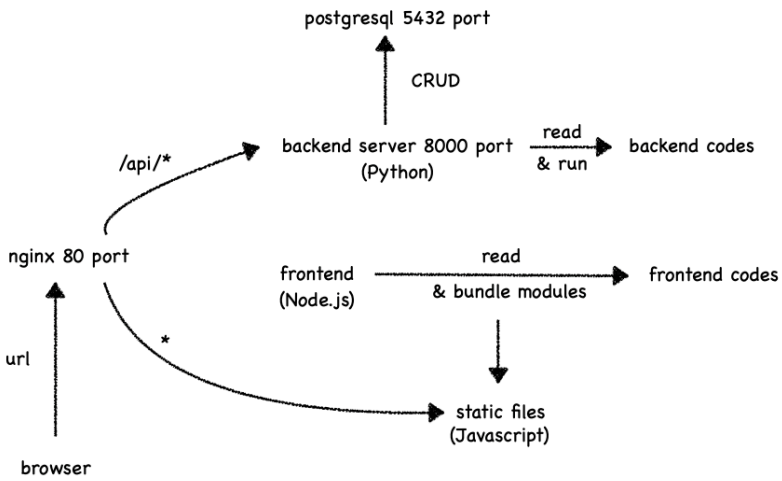

Figure 10: The toolchain

6.2 GitHub Repo

`https://github.com/CSC3170/new_project_server`

6.3 Build and run

```
$ git clone https://github.com/CSC3170/new_project_server.git
$ cd new_project_server/docker/prod
$ docker-compose up
```

6.4 Demo

`https://cuhko365-my.sharepoint.com/:v:/g/personal/119010103_link_cuhk_edu_cn/EeTBL9rkJIZKjxXAm4uxPnsBiBvHYcJ9mRIGwbuLUC4RoA?e=0f3cxe`

6.5 Contribution

Task	Member
ER & relational schema	赵松霖, 李韵文, 韩炅皓
Architecture design	胡海川
Frontend	邵晓雯, 胡海川
Backend	胡海川
Database	李韵文, 韩炅皓
Data analysis and data mining	赵松霖

References

Al-Jarf, R. (2022). Learning vocabulary in the app store by efl college students. *International Journal of Social Science and Human Research (IJSSHR)*, *5*(1), 216–225.

Kinkade, J. T. (2019). Building vocabulary and agency with an online app.

Setiawan, M. R., & Wiedarti, P. (2020). The effectiveness of quizlet application towards students' motivation in learning vocabulary. *Studies in English Language and Education*, *7*(1), 83–95.